

Lecture 3: Divide and Conquer, and Merge Sort

Harsh Parikh

hp537@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Bubble sort removes one record per call and pays about $n^2/2$ comparisons to order n records. Merge sort splits the list in half, orders each half by calling itself, and merges the two ordered halves in one pass. It halts because every call is on a shorter list, it is correct by strong induction, and it costs exactly $n \log_2 n - n + 1$ comparisons in the worst case when n is a power of two. No procedure that learns the order only by comparing records can beat a constant multiple of $n \log_2 n$.

AI use declaration. We used Claude Fable 5.1 to assemble this chapter from the five student scribe notes for Lecture 3 and to polish the writing. Every count, formula and trace was recomputed by `verify-lec03-scribe.py`, and every citation was checked against the published record. We did not verify page numbers inside the cited books.

1 Motivation

A clinic holds the records of n patients, each with a height recorded in centimetres, and the task is to arrange the records from shortest to tallest. Bubble sort orders the list by repeated sweeps. One sweep compares each neighbouring pair and exchanges the two when they are out of order, which carries the largest record to the end, and the next sweep repeats the work on the $n - 1$ records that remain. In the worst case the sweeps cost $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$ comparisons.

Five records cost 10 comparisons, a thousand cost 499,500, and a national register of ten million costs about 5×10^{13} , more than half a day for a machine performing one comparison every nanosecond. The cost grows with the square of n because each sweep removes only one record from the problem.

Splitting the list in half, ordering each half, and combining the two ordered halves costs about $n \log_2 n$ comparisons, which at ten million records is about 2.2×10^8 , a quarter of a second on the same machine.

2 Setup and Notation

Prerequisites. This chapter assumes summation notation, logarithms, and proof by induction. It states the form of induction it uses in Section 5 and assumes no prior knowledge of any sorting procedure.

Notation. Let A be a list holding n records, indexed from zero as $A[0], \dots, A[n-1]$, and write $A[i:j]$ for the slice $A[i], \dots, A[j-1]$, which holds $j-i$ records and is empty when $i \geq j$. Slices exclude their right end, as in Python, so $A[0:n]$ is the whole list, and $A[i:]$ is shorthand for $A[i:n]$. Write $|A|$ for the length of A , $\lfloor x \rfloor$ for the greatest integer not exceeding x , $\lceil x \rceil$ for the least integer not below x , and $\langle x \rangle + B$ for the list that starts with the record x and continues with the list B . The records are drawn from a *totally ordered set*, by which we mean that for any two records x and y exactly one of $x < y$, $x = y$, $x > y$ holds and the relation is transitive.

Definition 1 (The sorting problem). **Input:** a list A of n records drawn from a totally ordered set. **Output:** a *permutation*, that is a reordering, of those same n records, arranged so that $A[0] \leq A[1] \leq \dots \leq A[n-1]$. A list of zero or one records is sorted by definition.

Permutation means that the records that come out are the records that went in, with the same multiplicities, and Section 3 shows how easily a merge drops one.

Cost. We count *comparisons*, one for each test of a record against another, rather than seconds, which depend on the machine. Copying a record into an output list is not counted. We count *space* as the memory used beyond the input list. Unless stated otherwise a cost is the *worst case*, the largest count over all inputs of size n .

Growth. For a cost $T(n)$ and a comparison function $f(n)$, both non-negative, $T(n)$ is $O(f(n))$ if there exist $c > 0$ and $n_0 > 0$ such that $T(n) \leq c f(n)$ for all $n > n_0$. Likewise $T(n)$ is $\Omega(f(n))$ if $T(n) \geq c f(n)$ for all $n > n_0$, and $T(n)$ is $\Theta(f(n))$ if $c_1 f(n) \leq T(n) \leq c_2 f(n)$ for all $n > n_0$, for some $c_1, c_2 > 0$. In words, O bounds growth from above, Ω from below, and Θ both at once, with the constant absorbing every factor that depends on the machine.

3 Divide and Conquer

In this section, we build merge sort from its two parts: a split that produces two smaller instances of the sorting problem, and a merge that combines two sorted lists into one.

Algorithm 1 Merge sort, and the merge of two sorted lists

```
1: procedure MERGESORT( $A$ )
2:    $n \leftarrow |A|$ 
3:   if  $n \leq 1$  then
4:     return  $A$ 
5:   end if
6:    $A_1 \leftarrow \text{MERGESORT}(A[0:\lfloor n/2 \rfloor])$ 
7:    $A_2 \leftarrow \text{MERGESORT}(A[\lfloor n/2 \rfloor:n])$ 
8:   return MERGE( $A_1, A_2, 0, 0$ )
9: end procedure

10: procedure MERGE( $A_1, A_2, i, j$ ) ▷  $A_1$  and  $A_2$  are sorted
11:   if  $i = |A_1|$  then ▷ base case 1:  $A_1$  exhausted
12:     return  $A_2[j:]$ 
13:   end if
14:   if  $j = |A_2|$  then ▷ base case 2:  $A_2$  exhausted
15:     return  $A_1[i:]$ 
16:   end if
17:   if  $A_1[i] \leq A_2[j]$  then
18:     return  $\langle A_1[i] \rangle + \text{MERGE}(A_1, A_2, i + 1, j)$ 
19:   else
20:     return  $\langle A_2[j] \rangle + \text{MERGE}(A_1, A_2, i, j + 1)$ 
21:   end if
22: end procedure
```

The decomposition. Bubble sort reduces a problem of size n to one of size $n - 1$. Merge sort reduces it to two problems of sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, which differ by at most one, sorts each half by calling itself, and combines the two sorted halves. This pattern of dividing a problem into smaller instances of itself, conquering each by recursion, and combining the results is the *divide-and-conquer* paradigm (Cormen et al., 2022). The split point is $\lfloor n/2 \rfloor$ because an index must be an integer: for $n = 5$ the halves have lengths 2 and 3.

Merging. Two sorted lists A_1 and A_2 are combined by the image we acted out in class: two lines of people, each already in order of height, and at each step the shorter of the two people at the front steps forward into the output. The procedure keeps an index i into A_1 and an index j into A_2 , each marking the first record not yet placed. It compares $A_1[i]$ with $A_2[j]$, places the smaller, advances that one index, and calls itself. When either list is exhausted, the rest of the other list is already sorted and is placed wholesale, with no further comparisons. Algorithm 1 states both procedures, and each line transcribes directly into Python, with $A[:n//2]$ and $A[n//2:]$ for the halves and $[x] + B$ for $\langle x \rangle + B$.

Remark 1 (Two pointers, two base cases). `MERGE` advances two indices independently, and either list may run out first, so both exhaustion tests are needed. With only base case 1 the procedure reads $A_2[j]$ past the end whenever A_2 empties first, and a merge that stops as soon as either list empties, without copying the rest of the other, returns a sorted list that has lost records. The test is equality with the length because an index equal to $|A_1|$ is one position past the last record, so $i = |A_1|$ holds exactly when nothing is left.

4 Halting

In this section, we show that both procedures stop. The argument for each is the same: exhibit a non-negative integer computed from the arguments, a *measure*, that strictly decreases at every recursive call, since such a quantity cannot decrease forever.

Theorem 1 (Halting). *MERGE halts on every pair of finite sorted lists, and MERGESORT halts on every finite list.*

Proof. For `MERGE`(A_1, A_2, i, j) take the measure $d := (|A_1| - i) + (|A_2| - j)$, the number of records not yet placed. If either base case applies the procedure returns at once. Otherwise exactly one of i and j increases by one, so the recursive call has measure $d - 1$, and the calls reach a base case after at most d steps.

For `MERGESORT`(A) take the measure $n := |A|$. If $n \leq 1$ the procedure returns at once. Otherwise the two recursive calls are on lists of lengths $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, both at least 1 and at most $n - 1$ when $n \geq 2$, so both are strictly smaller than n . Each branch therefore reaches the base case, and the call to `MERGE` that follows halts by the first paragraph. $\square$

5 Correctness

In this section, we prove that the output is a sorted permutation of the input: the merge is proved correct on its own, and the sort is then proved correct assuming the merge, since assuming that a helper works because its caller works would be circular.

Strong induction. Let $P(n)$ be a claim about every non-negative integer n . *Strong induction* proves $P(n)$ for all n by proving the base cases and then proving that, whenever $P(k)$ holds for every $k < n$, $P(n)$ holds as well. Merge sort calls itself on sizes near $n/2$, so its proof needs a hypothesis covering every smaller size.

Lemma 2 (Correctness of merge). *Let A_1 and A_2 be sorted. Then `MERGE`(A_1, A_2, i, j) returns a sorted list containing exactly the records of $A_1[i:]$ and $A_2[j:]$, with multiplicity.*

Proof. Strong induction on $d := (|A_1| - i) + (|A_2| - j)$, the number of records not yet placed. *Base cases.* If $i = |A_1|$ the procedure returns $A_2[j:]$, which is sorted because A_2 is, and holds exactly the remaining records. The case $j = |A_2|$ is symmetric. *Inductive step.* Suppose neither list is exhausted, and assume the claim whenever fewer than d records remain. Let x be the smaller of $A_1[i]$ and $A_2[j]$, taking $A_1[i]$ on a tie. Every other record of $A_1[i:]$ is at least $A_1[i]$ and every record of $A_2[j:]$ is at least $A_2[j]$, because both lists are sorted, so x is the smallest record remaining in either. The procedure places x and calls itself with that one index advanced. The call has $d - 1$ records remaining, so by hypothesis it returns a sorted list of exactly the other remaining records, and placing the overall minimum in front of it gives a sorted list of all of them. $\square$

Theorem 3 (Correctness of merge sort). *For every finite list A , $\text{MERGESORT}(A)$ returns a sorted permutation of A .*

Proof. Strong induction on $n := |A|$. If $n \leq 1$ the list is sorted by Definition 1 and is returned unchanged. For $n \geq 2$, assume the claim for every length less than n . The halves $A[0:\lfloor n/2 \rfloor]$ and $A[\lfloor n/2 \rfloor:n]$ have lengths strictly less than n , so by hypothesis the two recursive calls return sorted permutations of them. Lemma 2 then says the merge returns a sorted list of exactly the records of both halves, and the halves partition A . $\square$

6 Cost

In this section, we count comparisons exactly, first for one merge and then for the whole procedure.

Lemma 4 (Cost of a merge). *Merging sorted lists of lengths a and b takes at most $a + b - 1$ comparisons, and exactly $a + b - 1$ when the largest record of the whole input and the second largest are in different lists.*

Proof. Each comparison places exactly one record and no record is placed twice, so at most $a + b$ comparisons are made. The procedure stops comparing as soon as one list is empty and copies the rest of the other, so the last record placed is copied rather than compared, and at most $a + b - 1$ comparisons are made. If the largest and second largest records are in different lists, every other record is smaller than both and is placed before either, so neither list empties until only the largest record remains, and $a + b - 1$ records are placed by comparison. $\square$

The recurrence. Let $n = 2^k$ for an integer $k \geq 0$, so that every half is a whole number of records, and let $M(n)$ be the number of comparisons on an input of size n when every merge takes its maximum. Sorting n records costs two sorts of $n/2$ records and one merge of two lists of $n/2$, which by Lemma 4 costs at most $n - 1$, so

$$M(1) := 0, \quad M(n) := 2M(n/2) + (n - 1) \quad (n > 1). \quad (1)$$

No input costs more than $M(n)$, and Remark 2 builds one that costs exactly $M(n)$, so $M(n)$ is the worst case.

The recursion tree. Draw the calls as a tree, with the call on n records at the root, its two calls on $n/2$ below it, four on $n/4$ below those, and the calls on single records as leaves (Cormen et al., 2022). Level i holds 2^i merges, each costing $n/2^i - 1$ by Lemma 4, so every level holds all n records and costs $n - 2^i$. There are $k = \log_2 n$ levels of merges, because that is how many times n can be halved before reaching one, and the leaves cost nothing, so the sum over $i = 0, \dots, k - 1$ is $nk - (2^k - 1)$. The tree supplies the guess and the theorem makes it exact.

Theorem 5 (Cost of merge sort). *Let $n = 2^k$ for an integer $k \geq 0$, and let M be as in Equation (1). Then $M(n) = n \log_2 n - n + 1$.*

Proof. Induction on k , with the claim $M(2^k) = k 2^k - 2^k + 1$. *Base case* ($k = 0$): $M(1) = 0 = 0 \cdot 1 - 1 + 1$. *Inductive step:* assume the claim at $k - 1$ for some $k \geq 1$. By Equation (1),

$$\begin{aligned} M(2^k) &= 2 M(2^{k-1}) + 2^k - 1 = 2[(k-1) 2^{k-1} - 2^{k-1} + 1] + 2^k - 1 \\ &= (k-1) 2^k - 2^k + 2 + 2^k - 1 = k 2^k - 2^k + 1, \end{aligned}$$

which is $n \log_2 n - n + 1$ at $n = 2^k$. □

Remark 2 (An input that reaches the worst case). For every $n = 2^k$ there is an input costing exactly $M(n)$. Build it by induction on k : at $n = 1$ any input works, and for $n > 1$ put the largest value in the left half and the second largest in the right, arranging each half in a worst-case order for size $n/2$, which exists by hypothesis. Both recursive sorts then cost $M(n/2)$ and the final merge costs $n - 1$ by Lemma 4.

Corollary 6 (Merge sort is $\Theta(n \log n)$). *$M(n)$ is $\Theta(n \log_2 n)$, with $c_1 = \frac{1}{2}$, $c_2 = 1$ and $n_0 = 1$.*

Proof. *Upper bound.* $M(n) = n \log_2 n - (n - 1) \leq n \log_2 n$ for all $n \geq 1$. *Lower bound.* The inequality $M(n) \geq \frac{1}{2} n \log_2 n$ rearranges to $\frac{1}{2} n \log_2 n \geq n - 1$, and dividing by n to $\log_2 n \geq 2 - 2/n$. At $n = 2$ both sides equal 1, and for $n \geq 4$ the left side is at least 2 while the right is below 2. □

Inside $\Theta(\cdot)$ the base of the logarithm is irrelevant, since $\log_a n = \log_b n / \log_b a$ and the constant is absorbed by c_1 and c_2 , so we write $\log n$ without one. In the exact count of Theorem 5 the base matters and is written.

General n . When n is not a power of two the recurrence becomes $M(n) = M(\lfloor n/2 \rfloor) + M(\lceil n/2 \rceil) + (n - 1)$, whose solution is

$$M(n) = n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1, \tag{2}$$

which agrees with Theorem 5 at every power of two and with exhaustive enumeration of every ordering for $n \leq 9$. Against bubble sort's $n(n-1)/2$ the gap opens slowly and then does not close: 8 against 10 at $n = 5$, 25 against 45 at $n = 10$, 49 against 120 at $n = 16$, 8,977 against 499,500 at $n = 1,000$, and a ratio of about 224,000 at $n = 10^7$.

Best case, and space. Merge sort splits and merges on every input, and on a sorted input each merge places all of A_1 by comparison against $A_2[0]$ and then copies A_2 , so the count satisfies $B(n) = 2B(n/2) + n/2$ with $B(1) = 0$, whose solution $B(n) = \frac{1}{2}n \log_2 n$ is 12 at $n = 8$ against the worst case of 17 and is still $\Theta(n \log n)$. A procedure *sorts in place* if only a constant number of records are ever stored outside the list (Cormen et al., 2022), and merge sort does not: each merge builds a new list, so the top-level merge alone holds n records beyond the input, for $\Theta(n)$ space.

7 Worked Example

Take the recorded heights of five patients, in centimetres, $A = \langle 171, 158, 180, 152, 163 \rangle$. The split point is $\lfloor 5/2 \rfloor = 2$, so the halves are $\langle 171, 158 \rangle$ and $\langle 180, 152, 163 \rangle$. The left half merges its two singletons in 1 comparison to $\langle 158, 171 \rangle$. The right half splits into $\langle 180 \rangle$ and $\langle 152, 163 \rangle$, the pair costs 1 comparison, and merging it with $\langle 180 \rangle$ compares 180 against 152 and then 163, after which the right list is exhausted and 180 is copied, for 2 comparisons and the result $\langle 152, 163, 180 \rangle$. The top-level merge of $L = \langle 158, 171 \rangle$ with $R = \langle 152, 163, 180 \rangle$ proceeds as follows.

step	compared	placed	output so far
1	158 and 152	152	$\langle 152 \rangle$
2	158 and 163	158	$\langle 152, 158 \rangle$
3	171 and 163	163	$\langle 152, 158, 163 \rangle$
4	171 and 180	171	$\langle 152, 158, 163, 171 \rangle$
5	none, L exhausted	180	$\langle 152, 158, 163, 171, 180 \rangle$

The top-level merge makes $4 = 2 + 3 - 1$ comparisons, the maximum of Lemma 4, because the largest record 180 is in R and the second largest 171 is in L . The total is $1 + 1 + 2 + 4 = 8$, which is Equation (2) at $n = 5$, and running the procedure on all 120 orderings of these five values gives none costing more, so this input attains the worst case. Both base cases of MERGE fired, case 1 at step 5 and case 2 in the merge of $\langle 180 \rangle$ with $\langle 152, 163 \rangle$.

8 Beyond the Lecture

Can any sorting procedure run in time proportional to n ? Not one of the kind built here. A *comparison sort* gains information about the records only by comparing pairs of them. Model such a procedure as a binary tree in which each internal node is one comparison with two outcomes and

each leaf is an output ordering. To be correct on every input of n distinct records the tree needs at least $n!$ leaves, and a binary tree of height h has at most 2^h leaves, so $2^h \geq n!$. At least $n/2$ of the factors in $n!$ are at least $n/2$, so $n! \geq (n/2)^{n/2}$ and $h \geq \frac{n}{2} \log_2 \frac{n}{2}$, which is $\Omega(n \log n)$. The height is the worst-case number of comparisons, so every comparison sort makes $\Omega(n \log n)$ comparisons in the worst case (Cormen et al., 2022), and merge sort is optimal among them up to a constant. A procedure that reads the records rather than comparing them escapes the bound: counting sort, given integer keys from a known range of size k , tallies occurrences in $\Theta(n + k)$ time (Cormen et al., 2022).

A sort is *stable* if records comparing equal keep their input order. MERGE takes from A_1 on a tie, and A_1 holds the earlier half of the input, so merge sort as written is stable, whereas the same code with $<$ in place of $\leq$ is not. Stability allows a table sorted by one key to be sorted again by another without disturbing the first. Merge sort is credited to John von Neumann in 1945 (Knuth, 1998), and the merge-based procedure that sorts lists in Python is Timsort (Peters, 2002). A three-way split, with a three-way merge costing at most $2(n - 1)$ comparisons, is still $\Theta(n \log n)$, since $\log_3 n$ is a constant multiple of $\log_2 n$.

9 Summary

Merge sort splits the list into two halves, sorts each by recursion, and merges the results in one pass. It halts because a measure strictly decreases at every recursive call. It is correct by strong induction on the length of the list. It costs exactly $n \log_2 n - n + 1$ comparisons in the worst case when n is a power of two, which is $\Theta(n \log n)$ against bubble sort's $\Theta(n^2)$. It pays for that speed with $\Theta(n)$ space, and no comparison sort can beat $\Theta(n \log n)$.

Further Reading

Cormen et al. (2022) give merge sort with a loop-invariant proof, the recursion-tree method, the decision-tree lower bound, and counting sort. Knuth (1998) covers the history of merging and exact counts, and Peters (2002) the merge-based sort used in Python.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.
- Peters, T. (2002). `listsort.txt`: Description of python's list sort. CPython source distribution, `Objects/listsort.txt`.